at how we can come up with an approach for automatic intent mining. One possible approach is to model this along the lines of a duplicate question-detection task, but allow for different entity values/slot values. All questions that are duplicates of each other have the same intent. We can then apply this relation transitively to obtain all the questions in each intent group. However, there is a problem associated with this approach. Let us assume that we have 1000 customer queries. Now, if we model this as duplicate question detection, we will need to consider all the possible pairs of questions, namely 499, 500. This can become non-scalable as the history of past conversations can be quite huge. How can we avoid this problem?

We propose to model the problem of automatic intent mining as a two-step process. In the first step, we identify clusters of semantically related customer queries in a conversation. Each of these identified clusters contains queries/questions whose intents are similar, but they may not be exactly equivalent. This helps to create smaller clusters of questions on which we can apply the pair-wise duplicate question check. Let us assume that we are given N conversational transcripts from a conversational corpus consisting of voice conversations between a customer and agent. Typically, the customer has a query, for which he requests the assistance of the agent. Let us also assume that we have separated out the customer utterances from agent utterances.

We prune the conversations to the first 'k' sentences of customer utterances, and each of these customer utterance documents becomes a single data point for the first phase. The value of 'k' is typically small, 1-3, and is determined heuristically based on the data set characteristics. Each of these extracted customer utterances is a document instance in our data set. We now want to perform intent mining on this set of documents.

Just as in any document processing task, we represent each document by a list of words contained in the document. The document word list is created by doing stop word removal, and stemming/lemmatisation. We then apply a linguistic rule based approach to identify intent indicators and intent holder words. This is by starting with an initial seed set of intent indicator words and then using them as seeds to extract patterns for intent holder words. We do this in a bootstrapped manner to obtain a list of corpus-specific intent indicator words and intent holder words.

Given an example utterance, "Can you please help me to open a new bank account?" the word 'help' is an intent indicator while 'open' is an intent holder. Since intent indicator words can deceive the intent mining algorithm due to the presence of predicates, we remove intent indicator words from the utterances in addition to the common stop words. We then build an utterance representation using universal sentence encodings. These are extensions of word level embeddings. These universal sentence encodings are intended to capture semantic relatedness well. We use these sentence encodings as vector inputs to a standard clustering algorithm such as kmeans, and obtain the semantic relatedness clusters in the first phase.

Once we create these semantic relatedness clusters, we can apply the duplicate question detection problem (modified to include unequal slot values) to each of these clusters, separately, since each of these clusters would typically be much smaller than the total number of document instances. This is the second phase where we try to identify the queries, which are exactly semantically equivalent. After running the duplicate question task, we have each pair of questions marked as duplicate or non-duplicate. We apply the duplicate relation transitively to create smaller clusters of semantically equivalent intent. We then aggregate all the non-duplicate questions (after applying transitive relation) into a separate cluster, which needs potential human intervention to identify intents.

Instead of modelling the second phase as a duplicate question detection problem, another possible approach is to consider applying a top-down clustering algorithm on these semantically related clusters. A top-down clustering algorithm applies divisive analysis to split a single cluster into two clusters, and repeats this process on each of the new clusters until the specified number of clusters is reached. In our case, we can just consider splitting the top level cluster into two — one cluster that contains semantically equivalent intents and another that is of semantically related but non-equivalent intents. The non-equivalent intent cluster will need human analysis for labelling. The semantically equivalent intent cluster has the intent identified automatically. This is the core approach behind automatic intent mining.

As mentioned earlier, we can consider different types of clustering algorithms for both the first and second phases in our approach. We can also consider different representations of the document instances – Bag of Words, neural sentence representations or universal sentence encodings. Of course, the key challenge is to define a suitable distance function for the clustering algorithm which can model semantic equivalence between document instances. We will discuss this topic in our next column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Here's wishing all our readers happy coding until next month. END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing.